# DSA0102 – Object Oriented Programming in C++

**Name:** Sanjay Manesh M

**Reg No:** 192511362

## Assignment 1

# 1. Introduction

The City Metro Smart-Card Fare System is designed to calculate passenger fares and manage smart-card balances based on the metro's fare rules. The metro network is divided into three zones: Inner, Middle and Outer, with base fares of ₹10, ₹20 and ₹30 respectively. A 20% surcharge is applied during peak hours, which are 8:00 AM to 10:00 AM and 5:00 PM to 8:00 PM. A journey is permitted only when the passenger has at least ₹50 remaining on the smart card after the fare is deducted. The system also considers concessions for eligible passengers, such as students and senior citizens.

The purpose of this assignment is to design, implement and validate the fare logic using C++ programming concepts. It explores object-oriented programming, data types, constants, operators, type conversions, control structures and input/output handling. The assignment also compares procedural and object-oriented approaches and examines how appropriate data types can improve the accuracy and reliability of fare calculations.

# 2. Objectives

1. To analyse the metro smart-card fare scenario and identify its main objects, rules and constraints.
2. To apply C++ concepts such as classes, objects, encapsulation, constants, enumerations, data types and operators.
3. To implement fare calculation for different zones and apply the peak-hour surcharge according to the specified time intervals.
4. To enforce the minimum remaining balance rule and prevent journeys that violate the fare conditions.
5. To understand type conversion, operator precedence, integer division and floating-point precision through practical programs.
6. To implement appropriate decision-making and looping structures for processing multiple journeys.
7. To compare procedural programming with object-oriented programming in terms of maintainability, data protection and code organization.

8. To design safe input handling and readable output reports, including recovery from invalid user input.
9. To validate the implementation using test cases covering normal journeys, peak-hour boundaries, insufficient balances and invalid inputs.
10. To compare alternative implementations using time complexity, space complexity and observed program results.
11. To recommend a suitable final design based on testing, correctness, maintainability and the expected growth in the number of journeys.

# Task 1 – Understand the Problem and Identify the Objects

## 1. Objective

The objective of this task is to analyse the Namma Metro smart-card fare system using object-oriented programming (OOP). The system must calculate the correct fare according to the travel zone and time, apply applicable concessions, and allow a journey only when the card has at least ₹50 remaining after fare deduction.

## 2. Identification of Objects

| Object | Data (attributes) | Behaviour (functions) | Justification |
|---|---|---|---|
| SmartCard | Card ID, balance | Check balance, check travel eligibility, deduct fare, recharge | Keeps the card balance and protects it from invalid deductions. |
| Passenger | Passenger ID, passenger type | Identify passenger type and applicable concession | Separates passenger information from card and fare calculations. |
| Journey | Starting details, destination zone, travel hour | Store journey details and validate journey information | Represents an individual metro journey. |
| Zone | Zone name and base fare | Identify the applicable zone fare | Organises the Inner, Middle and Outer zone categories. |
| FareTable | Base fares, peak surcharge, minimum balance | Find base fare and calculate the final fare | Keeps fare rules in one place, making future changes easier. |

These objects represent different responsibilities in the system. Keeping them separate improves readability, organisation and maintainability.

## 3. Procedural Programming vs Object-Oriented Programming

In a procedural approach, the system could use separate variables for card balance, passenger type and journey details, along with functions such as calculateFare(), isPeakHour() and deductFare(). Shared variables could be modified by different functions, making it harder to protect the balance rule.

In an object-oriented approach, data and related functions are grouped into classes such as SmartCard, Journey and FareTable. For example, the SmartCard class can keep the balance private and provide a function to deduct the fare only when the minimum-balance condition is satisfied.

Therefore, OOP provides better organisation and data protection for this system, although both approaches can have the same time and space complexity.

## 4. Basic Principles of OOP

**Encapsulation:** The card balance is kept private inside the SmartCard class. Other parts of the program cannot directly change it; they must use the class's functions. This helps prevent an invalid balance.

**Abstraction:** The main program can call a function such as canTravel(fare) without needing to know all the internal steps used to check the balance.

**Inheritance:** A general Passenger class could be extended by StudentPassenger and SeniorPassenger classes. Each could represent a passenger category with its applicable concession. This is a possible design, not a requirement to use inheritance everywhere.

**Polymorphism:** A common function such as calculateFare() could be implemented differently for regular passengers, students and senior citizens. The correct version could be used according to the passenger type. This would require a suitable class design and is an optional implementation choice.

## 5. Rules and Invariants

The following rules must be satisfied for a correct journey:

1. The valid zones are Inner, Middle and Outer, with base fares of ₹10, ₹20 and ₹30 respectively.
2. Peak hours are 08:00 to before 10:00 and 17:00 to before 20:00. Therefore, 10:00 and 20:00 are not peak hours.

3. The peak surcharge is 20% of the base fare.
4. The journey is approved only if the balance after fare deduction is at least ₹50.
5. A denied or invalid journey must not deduct money from the card.
6. The zone and travel hour must be valid before a fare is charged.
7. Any applicable student or senior-citizen concession must follow a defined concession policy.

The minimum-balance condition is important because the system must never allow a journey to leave less than ₹50 on the card.

## 6. Alternative Concept and Complexity Analysis

If the system used procedural programming, card balance and other details could be stored in variables, while free functions would calculate fares and check travel eligibility. In the OOP version, these responsibilities are grouped into classes.

For one journey, the program performs a fixed number of fare checks, comparisons and balance operations. Therefore, the time complexity is **O(1)** in both approaches. The memory required to store one card's balance and details is also **O(1)** in both approaches.

If a fare rule changes, a well-organised OOP design can keep the change within the relevant class or fare-rule component. A procedural design may require changes to several functions if the same rule has been repeated in multiple places. However, this depends on how well either version is organised.

## 7. Assumptions and Limitations

- The journey's destination zone determines the base fare.
- Peak-hour boundaries are interpreted as start-inclusive and end-exclusive.
- The fare must be deducted only when the remaining balance will be at least ₹50.
- The exact student and senior-citizen concession amounts are not specified in the assignment scenario, so they must be defined before implementing those concessions.
- This initial design covers the three specified zones. Future requirements, such as additional zones or new discount rules, may require changes.

## 8. Conclusion

The object-oriented approach is suitable for the metro smart-card system because it separates card management, passenger information, journey details and fare calculation. Encapsulation helps protect the balance rule, while abstraction and suitable use of inheritance and polymorphism can make the design easier to maintain. Both procedural and OOP approaches can process a journey in O(1) time, but a well-structured OOP design can make changes easier to manage.

# Task 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

The objective of this task is to define the fares, surcharge and minimum balance of the City Metro Smart-Card System in one place so that future changes can be made easily. In my C++ program, I used named constants for the Inner, Middle and Outer zone fares, the peak surcharge percentage and the minimum balance. The values are defined at the beginning of the program, which avoids repeating the same values throughout the code and makes the program easier to maintain.

C++ has five important categories of tokens: keywords, identifiers, constants, operators and punctuators. Keywords are reserved words with special meanings, such as int, return, switch and enum. Identifiers are names given to program elements, such as BASE_FARE_INNER, MIN_BALANCE and getBaseFare. Constants represent fixed values, such as 10, 20, 30 and 50 in this program. Operators perform operations, for example, = assigns a value, + adds values and == compares values. Punctuators include symbols such as semicolons, parentheses, braces and commas, which organize the structure of C++ code.

C++ provides #define, const and constexpr for working with fixed values. The #define directive creates a preprocessor macro that substitutes text before compilation. It does not provide normal C++ type checking or ordinary variable scope. The const keyword prevents a variable from being modified after initialization. The constexpr keyword is used when a value is required to be usable in a constant expression at compile time. I selected constexpr int for the fixed fare values because they are known in advance and do not change during program execution. This makes the fare definitions clear and allows the values to be reused consistently.

The metro system contains three zones, Inner, Middle and Outer. I represented them using enum class Zone { Inner, Middle, Outer };. An ordinary enum creates named integral constants, while enum class provides scoped names and stronger type safety. For example, Zone::Inner clearly identifies a zone and is safer than representing the same zone using an unexplained integer such as 1. The getBaseFare() function uses a switch statement to return the corresponding fare.

Meaningful identifier names improve readability. The constant names use uppercase letters and underscores, such as BASE_FARE_MIDDLE and PEAK_SURCHARGE_PERCENT, while the function name getBaseFare uses camel case. Identifiers must not contain spaces, must not begin

with a digit and must not use reserved C++ keywords. These conventions make the program easier to understand and maintain.

As an alternative, the fares and surcharge could be defined using #define macros. Both approaches can provide the required values, but macros perform text substitution and do not have the same type checking and scope rules as typed C++ constants. constexpr and enum class therefore provide a safer and clearer design. For this fixed set of three zones, selecting a fare using a switch takes constant time, O(1), in either approach. A macro normally occupies no storage as a variable, and a constexpr value usually requires no separate runtime storage either. Therefore, there is no guaranteed running-time improvement from choosing one approach over the other; the main advantage of the selected approach is type safety, readability and maintainability.

In conclusion, this task demonstrates the correct use of C++ tokens, identifiers, constants and enumerations. By defining fares and limits in one place and choosing constexpr and enum class, the program becomes easier to read, safer to modify and more suitable for future changes to the metro fare rules.

# Alternative approach – Macro

Example source code:

```
#define BASE_FARE_INNER 10

#define BASE_FARE_MIDDLE 20

#define BASE_FARE_OUTER 30

#define PEAK_SURCHARGE_PERCENT 20

#define MIN_BALANCE 50
```

| **Property** | **Macro approach** | **Your current approach** |
|---|---|---|
| Example | `#define BASE_FARE_INNER 10` | `constexpr int BASE_FARE_INNER = 10;` |
| Type checking | No normal type checking for the substitution | Typed C++ constant |
| Scope | Preprocessor substitution | Normal C++ scope rules |
| Memory | No variable storage for the macro | Usually no separate runtime storage for a compile-time constant |

| | | |
|---|---|---|
| Running time | O(1) for this fixed fare selection | O(1) for this fixed fare selection |
| Main advantage | Simple text substitution | Safer, clearer and easier to maintain |

# Task 3 – Choose Data Types and Check Type Compatibility

The purpose of this task is to select suitable C++ data types for the City Metro Smart-Card System and verify that the chosen types can represent the required values accurately. The system stores fares, card balances, journey times, zone information and journey counts. Choosing the correct data type is important because an unsuitable type can cause rounding errors, incorrect comparisons or integer overflow when the number of journeys increases.

Different data types are suitable for different kinds of information. The bool type is used for values that can be true or false, such as whether a journey occurs during peak hours. The char type can store a single character, such as a zone code. The int type is suitable for ordinary whole-number values such as the hour of a journey, provided the value stays within its range. The unsigned int type can represent non-negative counts such as the number of journeys at a station, but it should be used carefully because it cannot represent negative values and can cause unexpected results when mixed with signed integers. The long long type is selected for money stored as integer paise and for large accumulated totals. The float and double types represent floating-point numbers and are useful for measurements or approximate decimal calculations, but they are not the preferred types for exact money. The auto keyword allows the compiler to deduce a variable's type from its initial value; for example, auto journeyCount = 3000000; deduces an int on a typical implementation, while auto exampleFare = 1000LL; deduces long long.

The sizeof operator is used to measure how many bytes each type occupies on the system used for the experiment. The program displays the sizes of bool, char, int, unsigned int, long long, float and double. These values depend on the C++ implementation, so the actual output from the compiler should be recorded in the report. The <climits> header provides limits such as INT_MIN, INT_MAX, UINT_MAX, LLONG_MIN and LLONG_MAX. The <cfloat> header provides floating-point limits such as FLT_MAX and DBL_MAX. On many modern systems, int is 4 bytes and has a maximum value of 2,147,483,647, while long long is commonly 8 bytes and can store much larger integer values. The sizes and ranges printed by the program are used as the evidence for this task.

Money should be stored in integer paise rather than as double rupees because decimal fractions such as 0.1 cannot always be represented exactly in binary floating-point format. Repeated

additions can therefore produce a small difference between the stored floating-point total and the mathematically exact total. For example, a fare of Rs 10.01 can be represented exactly as 1001 paise using an integer. Adding integer paise values preserves the exact total as long as the selected integer type does not overflow. For this reason, long long is suitable for fare totals and balances that may become large. When displaying the result to the passenger, the paise value can be converted into rupees and paise for presentation.

C++ also applies type compatibility rules when different types are used in one expression. If two int values are divided, integer division is performed before the result is assigned to another type. For example, double result = 7 / 2; produces 3.0 because 7 / 2 is evaluated as integer division first. By contrast, double result = static_cast<double>(7) / 2; produces 3.5 because one operand is converted to double before division. When an integer such as 5 is assigned to a bool, the result is true, because any non-zero value converts to true and zero converts to false. These examples show why the type of an expression matters, not just the type of the variable receiving the result.

The assignment describes a growth from 20,000 journeys at one station to 500,000 journeys on a line and 3,000,000 journeys across the whole network each day. If every journey were charged the maximum base fare of 3000 paise, the daily total for 3,000,000 journeys would be 9,000,000,000 paise. This exceeds the common maximum value of a 32-bit signed int, which is 2,147,483,647. Therefore, an int is not sufficient for this worst-case total when the amount is stored in paise. A long long is a more suitable choice because it can represent the total without overflowing on the system used for this assignment. The program compares the calculated total with INT_MAX to demonstrate the problem and its resolution.

An alternative approach would store the fare as double rupees and add each fare to a floating-point total. Both adding a value to a total and adding integer paise take constant time, O(1), per addition. On a system where sizeof(double) and sizeof(long long) are both 8 bytes, each stored value also uses 8 bytes. However, the floating-point result may differ slightly from the exact total because of rounding. The program measures this difference after adding Rs 10.01 for 20,000, 500,000 and 3,000,000 journeys. The actual differences from the experiment should be recorded in the report rather than assumed in advance. The experiment illustrates why integer paise is the safer choice for exact fare calculations, while long long provides a much larger range than a typical int.

In conclusion, suitable data types help the metro system maintain accurate fares and balances as the number of journeys grows. The measured type sizes and ranges, the integer-division demonstration, and the daily collection calculation provide evidence for the choices. Using long long for money in integer paise avoids floating-point rounding in fare accumulation and reduces the risk of overflow for the network's daily total.

| Metro value | Recommended type | Why it is suitable |
|---|---|---|
| Peak-hour status | `bool` | Stores `true` or `false` |
| Zone character/code | `char` | Stores one character |
| Hour of journey | `int` | Stores values from 0 to 23 |
| Zone number, if represented numerically | `int` | Suitable for a small whole-number value |
| Fare and card balance | `long long` in paise | Exact integer money values with a large range |
| Journey count | `unsigned int` or `long long` | Non-negative count; `long long` gives more headroom |
| Approximate decimal measurement | `float` | Uses less storage on common systems, with lower precision |
| Floating-point comparison experiment | `double` | Higher precision than `float`, but still not exact for all decimals |
| Type inferred from initialization | `auto` | Lets the compiler deduce the type |

| Type | Size on your system | Limit to record |
|---|---|---|
| bool | Copy sizeof(bool) | false or true |
| char | Copy sizeof(char) | CHAR_MIN and CHAR_MAX |
| int | Copy sizeof(int) | INT_MIN and INT_MAX |
| unsigned int | Copy sizeof(unsigned int) | 0 to UINT_MAX |

| | | |
|---|---|---|
| long long | Copy sizeof(long long) | LLONG_MIN to LLONG_MAX |
| float | Copy sizeof(float) | -FLT_MAX to FLT_MAX |
| double | Copy sizeof(double) | -DBL_MAX to DBL_MAX |

# Task 4 – Control Implicit Conversions

The objective of this task is to identify implicit conversions in C++ and understand how they can affect the correctness of a program. An implicit conversion occurs when the compiler automatically changes a value from one data type to another. Although these conversions can make programming convenient, they may also produce unexpected results if the programmer does not understand the rules.

Integer division is one example of a conversion-related problem. In the expression double c = a / b;, where a is 7 and b is 2, both operands are integers. Therefore, integer division takes place first, producing 3. When this result is assigned to a double, it becomes 3.0, and the fractional part is already lost. In contrast, double d = static_cast<double>(a) / b; explicitly converts a to double before division. The result is 3.5. This demonstrates that converting the final result is different from converting an operand before an operation.

Character and Boolean conversions also occur in the program. In the expression 'A' + 2, the character is promoted to an integer before addition, producing the character C when stored back in a char. In the expression ch + 1, the character is promoted to an integer, so the program prints its numeric character code plus one. Similarly, assigning the integer 5 to a bool converts it to true, because non-zero integer values become true and zero becomes false. By default, an output stream displays Boolean values as 1 or 0; using boolalpha displays them as true or false.

The program also demonstrates the difference between signed and unsigned integers. A signed integer can represent negative values, but an unsigned integer cannot. When a signed negative value is compared with an unsigned value, the usual arithmetic conversions may convert the signed value to unsigned. Consequently, comparing -1 with an unsigned value of 1 can produce the unexpected result that -1 is greater than or equal to 1. When compiled with suitable warning options, such as -Wall -Wextra, GCC can warn about this signed-versus-unsigned comparison. The warning helps identify code that may behave differently from what the programmer expects.

Narrowing conversions can be prevented with brace initialization. For example, int x{3.7}; is rejected by the compiler because the conversion from 3.7 to an integer would discard the

fractional part. Ordinary assignment using int x = 3.7; permits the conversion but produces the value 3. Brace initialization is therefore useful when an unintended loss of information should be caught at compile time. Explicit conversion with static_cast can also make the programmer's intention clear, as demonstrated by converting a to double before division.

An alternative approach is to use explicit casts whenever a conversion is intended and brace initialization when variables are initialized. For example, static_cast<double>(a) / b makes the floating-point calculation clear, while int x{3.7}; causes a compilation error instead of silently discarding the decimal part. Explicit conversions do not automatically make every program correct, so the programmer must still choose a suitable target type and check whether the conversion preserves the required value.

For complexity analysis, the original and explicit-conversion approaches perform a fixed number of arithmetic and conversion operations. Therefore, the running time is O(1), and the additional space is O(1) for both approaches. Explicit casts and brace initialization do not normally create a meaningful running-time or memory advantage. Their main benefit is clarity and error prevention: some unintended conversions can be detected during compilation rather than appearing as unexpected results during execution.

In conclusion, implicit conversions are an important part of C++, but they must be understood and controlled. The examples demonstrate integer division, integer-to-double conversion, character promotion, integer-to-Boolean conversion and signed-to-unsigned comparison. Using static_cast where appropriate, brace initialization for safe initialization, and compiler warnings helps make programs more predictable and reliable.

# TASK 5 – Write Correct Expressions with the Right Precedence

In C++, operators are used to perform calculations, comparisons and logical operations. The order in which operators are evaluated is called **operator precedence**, while associativity determines the order of evaluation when operators have the same precedence. Correct use of these rules is important in the metro smart-card fare system because an incorrect expression can calculate the wrong fare or identify peak hours incorrectly.

The peak-hour condition is written as (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). The logical AND operator (&&) checks whether the hour belongs to one interval, and the logical OR operator (||) checks whether either interval is satisfied. Therefore, peak hours are from 8:00 AM to before 10:00 AM and from 5:00 PM to before 8:00 PM. The starting hour is included, but the ending hour is excluded. Parentheses make the expression easier to understand. The expression hour >= 8 && hour < 10 || hour >= 17 && hour < 20 gives the same result because && has higher precedence than ||, but using parentheses improves readability and reduces the chance of mistakes.

The base fare depends on the farthest zone travelled: Inner Zone costs Rs 10, Middle Zone costs Rs 20, and Outer Zone costs Rs 30. During peak hours, a 20% surcharge is applied. For example, the peak fare for the Middle Zone is Rs 24, calculated as 20 + (20 * 20 / 100). A conditional operator (?:) can select a value based on a condition, while compound assignment operators such as += combine an operation with assignment. For example, fare += surcharge adds the surcharge to the existing fare.

The increment operators x++ and ++x behave differently. The post-increment operator x++ uses the current value before incrementing the variable, whereas the pre-increment operator ++x increments the variable before using its value. In the program, int z = x++ + ++y; initially has x = 5 and y = 3. The expression uses 5 from x++ and 4 from ++y, so z becomes 9. After the expression, x becomes 6 and y becomes 4. The expression i = i++ should be avoided because it is confusing and may not produce the intended increment; its exact behaviour depends on the C++ language version. A separate increment statement such as i++ is clearer.

The division operator (/) performs integer division when both operands are integers, discarding the fractional part. The remainder operator (%) returns the remainder after division. For example, 10 / 3 gives 3 and 10 % 3 gives 1. Similarly, -10 / 3 gives -3 and -10 % 3 gives -1 because integer division truncates toward zero and the remainder follows the sign of the left operand.

The given program demonstrates operator precedence, increment operations, the conditional operator, multiplication, division, remainder and compound assignment. The first statement evaluates x + y * 2 as 11 because multiplication is performed before addition. The expression (x + y) * 2 gives 16 because the parentheses require addition to be performed first. The next statement demonstrates post-increment and pre-increment, producing z = 9, x = 6 and y = 4. The conditional expression (x > y ? x : y) returns 6 because x is greater than y. The expression x % 4 * 2 gives 4 because the remainder and multiplication operators have the same precedence and are evaluated from left to right.

The program then demonstrates positive and negative integer division and remainder. Finally, the for loop starts with i = 0, increases i by 3 each time, and adds each value to k. The values added are 0, 3, 6 and 9, giving a final value of 18. The loop stops when i reaches 12 because the condition i < 10 is no longer true.

The exact expected output of the program is:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

The output confirms that operator precedence and associativity affect the results of expressions. It also shows why increment operators, integer division and remainder must be used carefully. In the metro fare system, incorrect operator usage could lead to incorrect surcharge calculations or peak-hour decisions. A direct logical expression is simple and takes O(1) time and O(1) additional space. Alternatively, a 24-element Boolean lookup table can mark the peak hours and check peak[hour]. This method also takes O(1) time for a lookup but requires additional space for the table. Both approaches are efficient, but the direct logical expression is suitable for this small fixed set of peak-hour intervals because it is straightforward and easy to maintain.

The program was evaluated by predicting the output before execution and then compiling and running it to verify the results. The observed output should be compared with the predicted output, and any differences should be explained using the relevant C++ operator rules.

# TASK 6 – Design the Control Structures

The purpose of this task is to design the decision-making and looping logic required to calculate metro fares and process multiple journeys. The fare system has three zones: Inner, Middle and Outer, with base fares of Rs 10, Rs 20 and Rs 30 respectively. A 20% surcharge is applied during peak hours, and a journey is permitted only if at least Rs 50 remains on the smart card after the fare is deducted. The program uses switch, if–else, a for loop and functions to implement these rules.

The program defines the zone values using enum class Zone, which provides the three named values Inner, Middle and Outer. The BASE_PAISE array stores the fares as 1000, 2000 and 3000 paise. The constant PEAK_SURCHARGE_PERCENT stores the surcharge rate of 20%, while MIN_BALANCE_PAISE stores the minimum required balance of 5000 paise. Keeping these values in constants makes the program easier to maintain because fare rules can be revised in one place.

The fare() function calculates the fare for a journey. It uses a switch statement to select the correct base fare according to the zone. For the Inner Zone, the fare is 1000 paise; for the Middle Zone, it is 2000 paise; and for the Outer Zone, it is 3000 paise. The break statement at the end of each case prevents execution from continuing into the next case. The default case returns zero if the zone does not match any of the listed values. After selecting the base fare, an if statement checks whether the journey is during peak hours. If peak is true, the program calculates the 20% surcharge using integer arithmetic and adds it to the base fare. For example, the Outer Zone fare is 3000 paise, and its peak surcharge is 600 paise, giving a total fare of 3600 paise, or Rs 36.00.

The canTravel() function checks whether the journey is permitted. It subtracts the fare from the available balance and checks whether the remaining amount is at least 5000 paise. The function returns true if the condition is satisfied and false otherwise. This check prevents the program from deducting a fare when the journey would leave the card with less than the required minimum balance. The main function begins with a balance of 10000 paise, equivalent to Rs

100, and creates an array containing three journeys. Each journey contains its zone and whether it occurs during peak hours.

The for loop processes the journeys one by one. For each journey, it calls fare() to calculate the fare and then calls canTravel() to check whether the passenger is allowed to travel. If the condition is true, the fare is deducted from the balance and the fare and remaining balance are displayed. Otherwise, the program prints “Denied (low balance)” and leaves the balance unchanged. The loop is appropriate because the number of journeys in the array is known. A range-based for loop is used to access each journey directly without manually maintaining an array index.

The program uses <iomanip> to format the fare with two digits after the decimal point in the form of rupees and paise. The expression f / 100 obtains the whole-rupee part, while f % 100 obtains the paise part. setw(2) reserves two character positions, and setfill('0') adds a zero when the paise value has only one digit. After printing the paise portion, setfill(' ') restores the normal fill character.

The first journey is in the Inner Zone during off-peak hours. Its fare is 1000 paise, or Rs 10.00. After deducting the fare from the starting balance of Rs 100, the remaining balance is Rs 90.00, which is above the minimum balance requirement, so the journey is accepted. The second journey is in the Outer Zone during peak hours. Its base fare is Rs 30.00, and the 20% surcharge is Rs 6.00, making the total fare Rs 36.00. After this fare is deducted from Rs 90.00, the remaining balance is Rs 54.00. This journey is also accepted because the balance is still above Rs 50.00. The third journey is in the Middle Zone during off-peak hours, with a fare of Rs 20.00. Deducting this fare would leave Rs 34.00, which is below the required minimum balance of Rs 50.00. Therefore, the journey is denied and the balance remains Rs 54.00. The output confirms that the program calculates fares correctly, applies the peak surcharge and prevents a journey from violating the minimum-balance rule.

The assignment also requires testing the peak-hour boundaries. The correct condition for peak hours is (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). Therefore, 8:00 AM and 5:00 PM are peak times, while exactly 10:00 AM and exactly 8:00 PM are not peak times. These boundary values should be tested explicitly to ensure that the start of each interval is included and the end is excluded.

Different control structures are appropriate for different purposes in this program. A switch statement is suitable for selecting a base fare from a fixed set of zones. An if–else statement is suitable for checking peak-hour status and deciding whether a journey can be accepted. A conditional operator could also choose between two values, but if–else is clearer when several actions are required. A standard for loop is useful when processing a known number of journeys, a range-based for loop is convenient for iterating through an array, and a while loop is useful when processing continues until a condition becomes false, such as reading journeys until the user enters a stopping value. The continue statement could skip an invalid journey and move to the next one, while break could stop processing when a terminating condition is

reached. In the switch statement, break prevents accidental execution of subsequent cases, and default provides a response when no listed case matches.

An alternative design could store the base fares in an array indexed by the zone rather than selecting them with a switch statement. Both methods provide constant-time fare selection, or O(1), for a fixed number of zones. The array method uses extra storage for the fare values, while the switch method expresses each zone choice explicitly and can be easier to understand for a small fixed set of cases. If a fourth zone is added later, both approaches must be updated. The switch method needs another case, while the array method needs another fare entry and a reliable mapping between the zone and its array index. Overall, the selected control structures make the program readable, efficient and suitable for processing the metro's fare rules.

**Pseudo Code**

```
ALGORITHM CalculateFareAndProcessJourneys

CONSTANT MIN_BALANCE_PAISE = 5000

CONSTANT PEAK_SURCHARGE_PERCENT = 20

FUNCTION fare(zone, peak)
    CASE zone OF
        Inner: baseFare = 1000
        Middle: baseFare = 2000
        Outer: baseFare = 3000
        OTHERWISE: RETURN 0
    END CASE
    IF peak = TRUE THEN
        surcharge = baseFare * PEAK_SURCHARGE_PERCENT / 100
        baseFare = baseFare + surcharge
    END IF
    RETURN baseFare
END FUNCTION

FUNCTION canTravel(balance, journeyFare)
```

```
    IF balance - journeyFare >= MIN_BALANCE_PAISE THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF
END FUNCTION
BEGIN
    balance = 10000
    FOR each trip in trips
        journeyFare = fare(trip.zone, trip.peak)
        IF canTravel(balance, journeyFare) THEN
            balance = balance - journeyFare
            DISPLAY journeyFare and balance
        ELSE
            DISPLAY "Denied (low balance)"
            DISPLAY balance
        END IF
    END FOR
END
```

# TASK 7 – From Procedural to Object-Oriented

The purpose of this task is to implement the metro smart-card fare logic using both procedural programming and object-oriented programming and compare their advantages and limitations. In the procedural approach, the program uses separate variables and free functions to calculate fares, check whether a passenger can travel, and update the card balance. In the object-oriented approach, a `SmartCard` class stores the balance privately and provides member functions to check travel eligibility, deduct fares and retrieve the balance. Both implementations use the same fare values and minimum-balance rule so their behaviour can be compared fairly.

The program defines the constants `MIN_BALANCE`, `INNER_FARE`, `MIDDLE_FARE` and `OUTER_FARE`. The values are stored in integer paise to avoid floating-point rounding problems. The minimum permitted remaining balance is 5000 paise, equivalent to Rs 50. The three base fares are 1000 paise for the Inner Zone, 2000 paise for the Middle Zone and 3000 paise for the Outer Zone. These constants avoid repeating the same numerical values throughout the program and make fare revisions easier.

In the procedural version, the function `getFare()` uses a `switch` statement to return the fare corresponding to a zone number. Zone 1 represents the Inner Zone, zone 2 represents the Middle Zone, and zone 3 represents the Outer Zone. If an invalid zone number is supplied, the function returns zero. The function `proceduralCanTravel()` checks that the fare is valid and that deducting it will leave at least the minimum balance. The function `runProcedural()` stores the card balance in a separate variable, calls the fare and eligibility functions, and deducts the fare only when the journey is permitted. This approach is straightforward, but the balance and the rules governing it are maintained separately. Other parts of a larger program could accidentally change the balance without using the checking function.

In the object-oriented version, the `SmartCard` class contains a private data member named `balance`. Because the member is private, code outside the class cannot directly change it. The constructor initializes the card balance when a `SmartCard` object is created. The member function `canTravel()` checks whether the balance remaining after a proposed fare would be at least Rs 50. The member function `deduct()` calls this check and subtracts the fare only when the journey is permitted. It returns `true` when the deduction succeeds and `false` otherwise. The `getBalance()` function provides read-only access to the current balance. These functions group the balance and the rules that control it in one class, demonstrating encapsulation.

The class helps preserve the minimum-balance rule because the program must use the class's public functions to perform a deduction, rather than directly modifying the private balance. For example, after the card balance reaches Rs 60, a Middle Zone fare of Rs 20 would leave only Rs 40. Since Rs 40 is below the minimum required balance of Rs 50, `canTravel()` returns `false` and `deduct()` does not subtract the fare. The balance remains Rs 60. This prevents an invalid deduction through the class's normal public interface. The constructor should also be given a valid initial balance, and any future recharge function should validate its input.

The program begins each version with a balance of Rs 100. The first journey uses the Inner Zone fare of Rs 10. The remaining balance becomes Rs 90, so the journey is accepted. The second journey uses the Outer Zone fare of Rs 30. The balance becomes Rs 60, which still satisfies the minimum-balance rule, so this journey is also accepted. The third journey uses the Middle Zone fare of Rs 20. Deducting this fare would leave Rs 40, so the journey is denied and the balance stays at Rs 60. The procedural and object-oriented versions produce the same results, demonstrating that both implement the same fare and balance rules.

The expected output is:

```
PROCEDURAL VERSION
Journey accepted. Balance Rs 90
Journey accepted. Balance Rs 60
Journey denied. Balance Rs 60

OBJECT-ORIENTED VERSION
Journey accepted. Balance Rs 90
Journey accepted. Balance Rs 60
Journey denied. Balance Rs 60
```

The output shows that both approaches calculate the same balances and reject the third journey. The main difference is how the data and rules are organized. The procedural version relies on separate variables and functions, while the object-oriented version keeps the balance private and places the eligibility and deduction operations inside the `SmartCard` class. If the minimum balance changes from Rs 50 to another amount, the procedural version requires the shared rule and its uses to be reviewed. In the object-oriented version, the rule can be maintained centrally and applied consistently through the class functions.

An alternative object-oriented design could make the balance a public data member. This would allow other parts of the program to change it directly, for example by assigning a negative value or subtracting a fare without checking the minimum balance. Such a design could break the rule that the balance must never become invalid through a journey deduction. Making the balance private and providing controlled member functions is therefore safer. Public and private access do not significantly change the basic time or space complexity: a balance check and deduction each take O(1) time, and storing a card balance requires O(1) space per card in either design. The object-oriented version is preferable because it provides stronger protection for the balance rule and makes future changes easier to manage.

# TASK 8 – Read and Print Data Safely

The objective of this task is to design safe input and clear output for a city metro smart-card gate program. The program accepts a zone number and journey hour, calculates the fare, checks the minimum balance requirement, and displays a journey report. The program uses whole rupee amounts: ₹10 for the Inner zone, ₹20 for the Middle zone, and ₹30 for the Outer zone. A surcharge of 20% is applied during peak hours, defined as 08:00–10:00 and 17:00–20:00. A journey is accepted only when at least ₹50 remains after deducting the fare.

The program reads values using cin. When a user enters letters instead of a number, the input operation fails and cin.fail() becomes true. The program handles this situation by calling cin.clear() to reset the error state and cin.ignore(numeric_limits<streamsize>::max(), '\n') to discard the invalid input remaining in the input buffer. It then asks the user to enter the value

again. The program also checks the permitted range for the zone and hour, preventing invalid numerical values from being used in fare calculations.

The getFare() function uses a switch statement to select the base fare. The peak-hour condition uses logical operators to identify the correct time intervals. If the journey is during peak hours, the program calculates the 20% surcharge using integer arithmetic. Before deducting the fare, it checks whether the remaining balance will be at least ₹50. If the condition is not satisfied, the journey is denied and the balance remains unchanged.

The program uses <iomanip> to improve the presentation of the output. The setw() manipulator allocates a field width for table columns, while setfill() specifies the character used to fill unused field positions. The fixed and setprecision() manipulators control the display of floating-point values. Since this version stores fares and balances as integer rupees, it displays whole amounts without decimal fractions. The report displays the journey number, zone, fare, remaining balance, total fare paid, and final balance.

In the sample test, the card starts with ₹100. The first journey is in the Inner zone during peak hours, so the fare is ₹12 and the balance becomes ₹88. The second journey is in the Outer zone during peak hours, so the fare is ₹36 and the balance becomes ₹52. The third journey costs ₹20, which would reduce the balance to ₹32. Because this is below the minimum required balance of ₹50, the journey is denied. The total fare paid is ₹48 and the final balance is ₹52. When letters are entered for a numerical input, the program displays an error message, clears the input failure, discards the invalid characters, and requests the value again.

As an alternative, each input line could be read using getline() and converted to an integer using stoi() inside a try/catch block. If the input cannot be converted to an integer, stoi() can throw an exception, which the program can catch to display an error and request new input. Both approaches require O(L) time to process an input line of length L in the general case and O(L) temporary space for the input line. Direct cin input is simple and works well for numeric input, but recovery requires clearing the stream state and discarding the invalid characters. The getline() and stoi() approach provides line-based validation and makes it easier to reject unwanted trailing characters when additional checks are added. Both approaches can provide safe input handling when implemented correctly.

The program processes a fixed number of journeys, with each journey requiring a constant number of fare calculations and checks. Therefore, the time complexity is O(1) per journey and O(n) for n journeys. The program uses O(1) additional working space per journey, excluding input buffering and the output produced. The result demonstrates safe recovery from invalid input, validation of numerical ranges, enforcement of the minimum-balance rule, and clear reporting of fare transactions.

# TASK 9 – Implement and Test the Fare Logic

The objective of this task is to implement and test the fare calculation logic for the City Metro Smart-Card System. The program uses whole rupee amounts, with base fares of ₹10 for the Inner zone, ₹20 for the Middle zone and ₹30 for the Outer zone. A surcharge of 20% is applied during peak hours, which are from 08:00 to before 10:00 and from 17:00 to before 20:00. A journey is accepted only when the remaining balance after fare deduction is at least ₹50.

The getFare() function uses a switch statement to select the base fare according to the zone. The peak-hour condition checks both specified time intervals. At 08:00 and 17:00, the surcharge applies; at 10:00 and 20:00, the surcharge does not apply. The surcharge is calculated using integer arithmetic. The program checks the remaining balance before deducting the fare, so a journey that would leave less than ₹50 is denied without changing the balance.

The input function validates the entered values. If the operator enters letters instead of a number, the program uses cin.clear() to reset the input error and cin.ignore() to discard the invalid characters before requesting the value again. An invalid zone number outside the range 1–3 is rejected by the input validation. Negative recharge amounts are also rejected, and the card balance remains unchanged. These checks help prevent invalid data from affecting the fare calculation.

Testing includes normal journeys in all three zones, both peak and off-peak journeys, the four peak-hour boundaries, invalid inputs, negative recharge and the minimum-balance condition. For the five-journey example, the starting balance is ₹100 and the recharge is ₹0. Journey 1 costs ₹10 and leaves ₹90. Journey 2 is a Middle-zone peak journey costing ₹24 and leaves ₹66. Journey 3 is an Outer-zone peak journey costing ₹36, but it is denied because the remaining balance would be ₹30. Journey 4 costs ₹10 at 10:00 and leaves ₹56. Journey 5 costs ₹10 at 20:00 but is denied because the remaining balance would be ₹46. Therefore, the total deducted fare is ₹44 and the final balance is ₹56.

As an alternative to separate hand-written test cases, a table of test inputs and expected outputs can be stored in an array of structures and processed using a loop. Each entry can contain the zone, hour, starting balance, expected fare and expected result. The program calculates the actual result and compares it with the expected result. This approach makes it easier to add new tests, especially boundary cases and invalid inputs, and to identify a failed test. If there are t test cases, the total time complexity is O(t), assuming constant-time processing per case. The test table requires O(t) space. The individual fare calculation and balance check each take O(1) time. A table-based test design is easier to extend, while separate hand-written checks may be simpler for a very small number of cases.

The test results demonstrate the importance of validating input, checking peak-hour boundaries and verifying the balance before deducting a fare. The final implementation prevents negative recharges, handles invalid numerical input and maintains the minimum-balance rule. The actual execution results and screenshots should be included as evidence in the assignment report.

# TASK 10 – Compare Expectations with Execution

The purpose of this task is to compare fare totals at station, line and whole-network levels and examine the behaviour of different C++ numeric data types. The program uses 20,000 journeys per day for one station, 500,000 for one line and 3,000,000 for the entire network. It calculates totals using `double`, `int` and `long long` and displays a floating-point experiment.

For the demonstration, the fare used for the `double` calculation is ₹10.01, whereas the integer calculation uses ₹10. Therefore, the totals are not equal because the fare values differ. For 20,000 journeys, the totals are ₹200,200.00 and ₹200,000. For 500,000 journeys, they are ₹5,005,000.00 and ₹5,000,000. For 3,000,000 journeys, they are ₹30,030,000.00 and ₹30,000,000. This difference is caused by the different fares and should not be described as floating-point error.

The program checks the maximum value of `int` using `INT_MAX` from the `<climits>` header. On a typical system with a 32-bit `int`, the maximum is 2,147,483,647. The network total of ₹30,000,000 fits within this range, so `int` does not overflow in this rupee-only example. However, storing money in a smaller unit would create larger numeric totals and could cause an `int` overflow. A wider integer type such as `long long` provides a larger range for accumulating totals.

The floating-point experiment compares repeated addition with direct multiplication. Both results may display as ₹10,010.00 when rounded to two decimal places. This displayed result does not prove that the internal values are exactly equal, because binary floating-point cannot represent every decimal fraction precisely. A small difference may be hidden by the output formatting.

The alternative approach is to accumulate totals using `int`, `long long` or `double`. Integer types provide exact arithmetic for values that fit within their ranges, while `double` can represent fractional rupee amounts but may introduce small rounding errors. Each addition or multiplication takes constant time, O(1), and each numeric variable uses constant space, O(1). Processing n journeys takes O(n) time and uses O(1) extra space when only running totals are stored.

The program is compiled with `-Wall -Wextra` to enable common compiler warnings. Any warnings observed during execution should be recorded with their exact messages and explained. If no warnings are generated, that should also be reported honestly. For a production metro system, the money representation should follow the assignment specification, and integer-based monetary storage is generally preferable when exact fare accounting is required.

# TASK 11 – Recommend & Defend the Final Design

The recommended design for the city metro smart-card system uses centralized fare constants, an `enum class` for zones, separate functions for fare calculation and peak-hour checking, and a `SmartCard` class that protects the passenger's balance. These choices make the program

easier to understand, test and maintain. The fare values are stored in one place, reducing the possibility of inconsistent values being used in different parts of the program.

The program uses integer rupees for this demonstration, an enumeration for the zone, and integer values for the hour and balance. The `isPeakHour()` function applies the peak periods from 8:00 to before 10:00 and from 17:00 to before 20:00. The `calculateFare()` function adds the 20% surcharge when the journey takes place during a peak period. The `SmartCard` class stores the balance as a private data member and provides `canTravel()`, `deduct()` and `getBalance()` functions. This prevents other parts of the program from directly changing the balance and helps enforce the minimum balance rule.

The sample execution begins with a balance of ₹100. The first journey has a fare of ₹12 and leaves ₹88. The second journey has a fare of ₹36 and leaves ₹52. The third journey is denied because deducting ₹20 would leave only ₹32, below the required minimum of ₹50. The final balance therefore remains ₹52. These results demonstrate the peak surcharge and balance-protection rules.

For future fare revisions, the constants can be updated in one place. An alternative is to load fares from a configuration file when the program starts. Loading a table of z zone fares takes O(z) time and O(z) space, while retrieving an individual fare from the table takes O(1) time. A configuration file makes fare changes more convenient because the program need not be recompiled for every fare revision, but it requires validation of the file and handling of missing or incorrect values.

The current design is suitable as a basic educational model, but it has limitations. It uses whole rupees for simplicity, processes a small fixed set of sample journeys and does not save transactions permanently. A production system would require exact monetary representation consistent with the fare specification, reliable transaction records, safe handling of simultaneous card operations, input validation, security and recovery procedures. The design can process a large number of journeys in O(n) time when each journey takes constant time, but a real deployment would require performance and reliability testing. The recommendation is to retain the modular, object-oriented design and extend it with validated configuration, exact monetary storage and persistent transaction handling before production use.

# TASK 12 – Re-evaluate and Adapt the Design

The original metro fare system supported three zones: Inner, Middle and Outer. The changed requirements introduce a fourth Airport zone with a base fare of ₹60, a 10% weekend discount that cannot be combined with the peak surcharge, and a monthly pass that permits unlimited journeys in two selected zones. These changes affect the zone enumeration, fare constants, fare calculation logic, card processing and test cases.

The design is modified by adding `Airport` to the `Zone` enumeration and defining `AIRPORT_FARE` as ₹60. The `getBaseFare()` function is updated with an Airport case.

The weekend rule is implemented separately from the peak-hour rule. When a journey takes place on a weekend, the program applies the 10% discount. Otherwise, if the journey occurs during peak hours, it applies the 20% surcharge. This prevents both rules from being applied to the same journey.

A `MonthlyPass` class is introduced to store the two zones covered by the pass. Its `covers()` function checks whether a journey is within the covered zones. If the pass covers the selected zone, the journey is accepted without deducting a fare from the stored-value card. Otherwise, the normal fare is calculated and the `SmartCard` class checks whether the balance will remain at least ₹50 after deduction.

The test program begins with a balance of ₹150 and a pass covering Inner and Middle. The first journey is in Inner during peak hours, but the pass covers the zone, so the fare is ₹0. The second journey is in Outer on a weekend. The 10% discount reduces its base fare from ₹30 to ₹27, leaving a balance of ₹123. The third journey is in Airport on a weekday at 18:00. The 20% peak surcharge increases its fare from ₹60 to ₹72. The remaining balance is ₹51, so the journey is accepted because the minimum balance rule is satisfied.

An alternative design uses a fare table indexed by zone instead of a switch statement. For a fixed number of zones, fare retrieval takes O(1) time. A Boolean array can also represent the zones covered by the monthly pass, allowing a coverage check in O(1) time. Processing n journeys takes O(n) time and uses O(1) additional space for fixed-size data structures.

The modified design retains the original object-oriented structure because the `SmartCard` class continues to protect the balance and the new `MonthlyPass` class separates pass-related behaviour from stored-value payment. The main limitations are that this is a small demonstration using whole rupees, a fixed number of zones and a simple in-memory pass. A real implementation would need exact monetary storage, monthly pass expiry and validity checks, persistent transaction records, robust input validation and security. Based on the demonstrated examples, the design can accommodate the new requirements without a complete redesign, but further tests would be required before real-world deployment.